

TRIMBLE INTERVIEW EXPERIENCE

There were five rounds in total, though due to a few external reasons, I had to attend two technical rounds, making it six rounds for me personally.

1. First Round: Coding (Codility - AI-assisted)
2. Second Round: AI Round (Brownfield and Greenfield Engineering)
3. Third Round: Technical HR (Based on the previous round and standard technical topics)
4. Fourth Round: Managerial HR
5. Fifth Round: PX HR (People Experience)

FIRST ROUND: CODING

There were two questions, both focused on DSA. There weren't many language-specific quirks to worry about.

In this round, you use an AI coding assistant to write the solution. Your prompts, the final solution, and your overall process are all monitored.

Things you should not do: Don't ask for the full code or direct solutions.

Here is my suggested flow: try brute force first, then explain the problem and your approach to the AI. Ask it to identify logical mistakes or evaluate your current trajectory. Finally, write the optimal solution. For example, instead of asking it to write a program to find frequencies, ask what data structure you can use for frequency measurement in the most optimal way.

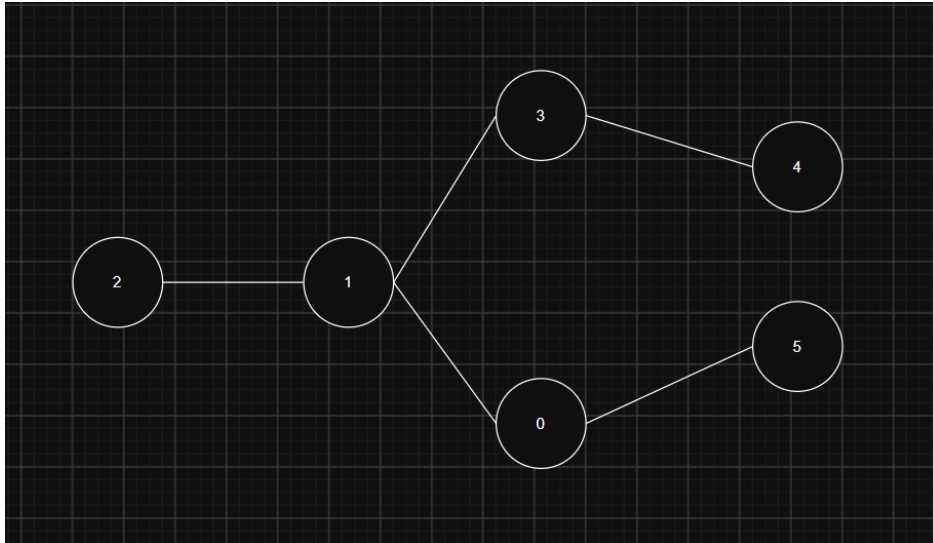
Spend a few minutes thoroughly reading the questions.

These are the questions I received:

1. Find all the shortest distances to any district from a given set of hospitals and return the longest distance among those minimum distances. It is an undirected graph. (This is a clearer version of the question, which was originally quite narrative and hard to understand, so use the AI to clarify any doubts.)

Input: A[], B[], H[], N. Here, A and B contain nodes such that A[i] and B[i] are connected (resembling an adjacency list/matrix). N is the number of nodes and H is the array of nodes that are hospitals.

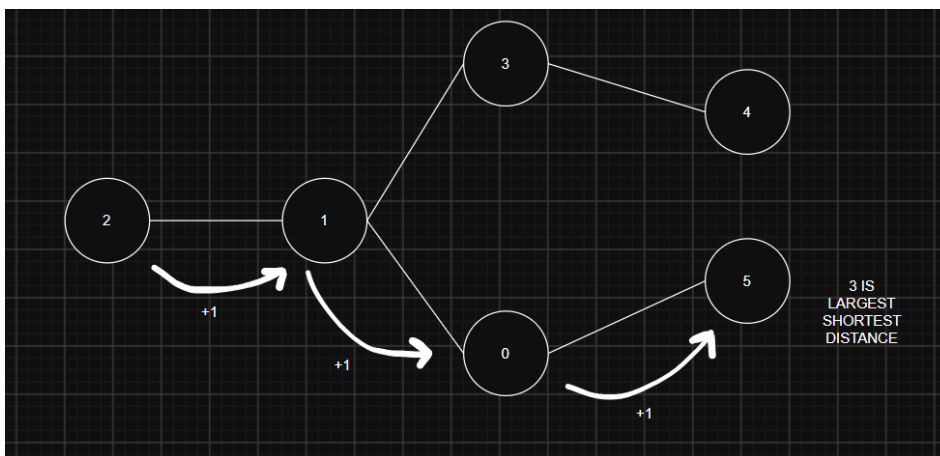
I initially thought of storing results using DFS, but after understanding the requirements, I realized it was a multi-source BFS problem. Use a Queue to perform a BFS starting from all H nodes simultaneously.



The above is a sample provided to us:

Input: $n = 6$, $A = \{2, 1, 1, 3, 0\}$, $B = \{1, 0, 3, 4, 5\}$, $H = \{2, 4\}$

Output: 3

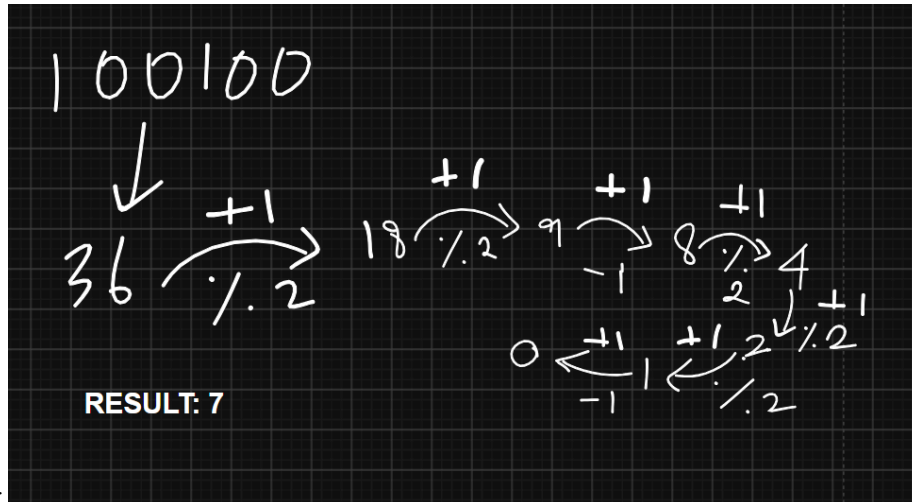


Try solving this on your own first, then use GPT or another AI tool to simulate this round and benefit from the practice.

2. You will be given a binary string. Your goal is to find the total number of operations it takes to reduce this number to zero. The rules are:
- If the number is even, divide it by 2.
 - If the number is odd, subtract 1 from it.

Input: "100100"

Output: 7



Explanation:

You might think that converting it to an integer will work, but it doesn't; TLE (Time Limit Exceeded) was the result for that approach.

Instead, you can use binary concepts: count the number of 1s in the string and the number of bits after the first 1 (the MSB, which isn't included in the count). Sum them to find the total operations.

For example, in "100100", the number of ones = 2, and the number of bits after the MSB = 5. The total is 7.

The number of bits after the MSB shows how many times you divide by 2, and the total count of ones shows how many times the number becomes odd.

Use AI wisely during this round. Research prompt engineering and make sure not to ask for solutions directly.

Here is the resource they shared with us: <https://www.promptingguide.ai/>

ROUND 2: AI EVALUATION

This was probably the most unusual interview round I've attended. It wasn't a traditional coding round, nor a pure system design interview. The entire evaluation revolved around how effectively you could use AI as an engineering tool.

The round was split into two parts, with **25 minutes allocated for each**.

Part A – Brownfield Engineering (25 mins)

The first part provided an existing codebase with intentionally hidden bugs, a walkthrough document, and failing tests. The goal wasn't to build something new—it was to understand an unfamiliar project quickly, trace the execution flow, identify root causes, and fix the issues using AI tools.

On paper, 25 minutes sounded manageable, but the reality was very different.

It took me nearly **45 minutes** just to get through the Brownfield task. This wasn't because the bugs were impossibly difficult, but because everything that could go wrong did.

Specifically:

- My laptop crashed midway.
- Cursor became unstable.
- Antigravity (tooling) wasn't cooperating.
- My GitHub Copilot tokens ran out.
- I ended up juggling between ChatGPT, DeepSeek, and Perplexity while manually running tests after every change.

At one point, I wasn't debugging the project anymore, I was debugging my own tooling.

Ironically, switching between multiple AI models did help later because each one had different strengths. Cursor was great for repository understanding, ChatGPT helped reason through architecture and regression risks, DeepSeek was useful for implementation ideas, and Perplexity helped verify assumptions quickly.

However, I wouldn't recommend this approach. The context switching is exhausting, and every tool has a slightly different understanding of the project. You spend more time syncing context than solving the actual problem. It's high risk for low reward. If possible, pick one primary tool and stick with it.

The context switching alone is exhausting, and every tool has a slightly different understanding of the project. You spend more time syncing context than solving the actual problem.

The actual engineering workflow mattered much more than writing code. My prompt sequence looked something like this:

- Understand the repository architecture.
- Identify the important files.
- Trace the request execution path.
- Map failing tests to affected files.
- Find root causes instead of symptoms.
- Suggest the smallest possible fixes with the least regression risk.

That last part is crucial. Don't ask AI to "fix everything." Ask it to suggest the **minimum changes required**. Large refactors during an interview are almost always a bad idea unless explicitly requested.

The interviewers weren't just interested in the final code; they wanted to see *how* I used AI. Every prompt I wrote was visible, and I had to explain why I asked certain questions and why I accepted or rejected specific suggestions.

Don't blindly copy AI output. Understand it, challenge it, and verify it by running tests. AI is like a very fast intern, but you are still expected to be the engineer.

Part B – Greenfield Engineering

The second half shifted from debugging to designing a system from scratch. We were given a problem statement for a B2B License Management System and had to design the complete architecture, database schema, REST APIs, concurrency handling, and supporting diagrams.

AI generated a detailed design document within seconds. However, because the Brownfield task took longer than expected, I walked into the Greenfield round with **only about 10 minutes left**.

There was no realistic way I could read, verify, and understand several pages of AI-generated system design in that time. I made a conscious decision: I read only the first couple of lines to ensure the approach matched the problem statement, checked that the overall structure looked reasonable, and submitted it. I wasn't going to pretend I understood a document I hadn't properly read.

If the interviewer had pointed to a random section and asked me to explain it, I wouldn't have been able to defend it confidently. Later, during the technical interview, they asked me a simple question: **"How much of the design did you actually work on? Did you actually have the tokens to complete it?"**

I told them the truth. I explained that I had only read the beginning because I had barely ten minutes left after the Brownfield round, and I didn't think it was right to claim I understood an AI-generated document I hadn't reviewed.

That honest answer unexpectedly changed the entire direction of the interview. What happened next taught me something far more valuable than system design itself.

Here are the links for the project materials: [📁 TRIMBLE_STUFFS](#)

ROUND 3: TECHNICAL HR

In this round, I explained everything about the Brownfield task: the prompts I used, why I used them, the bugs I encountered, and how I fixed them. They probed the technical concepts related to those bugs. For example, if there was a status code error, they asked about status codes; if there was a bearer token issue, they asked about tokens and authentication. It's important to understand the architecture even if the tech stack is new. I had no prior experience with "SQLAlchemy," but I managed by understanding the flow.

Later, they asked fundamental data structure questions and tested my knowledge of system design.

The technical interviewer drew a car and gave me a large use case, asking me to design a complete system for it—essentially OOAD (Object-Oriented Analysis and Design).

Here is a resource I used to prepare (the next 10 minutes of the interview revolved around LLD): [What is LLD?](#) Learning SE terms like "regression risk" helps significantly during these discussions.

After that, they revisited the Greenfield project. When I explained what happened, they handed me a sheet of paper and gave me 30 minutes to use any AI tool to actually work on the design.

I examined the architectural decisions the AI had made in the original document and used another AI tool to question those decisions. It took some time, but it was thorough.

Things they expected: a detailed REST API construct, a detailed DB schema, a UML diagram (HLD), and a deep understanding of the project flow. Always consider edge cases.

Think of edge cases too

ROUND 4: MANAGERIAL HR

The managerial round was straightforward. They were looking for specific behaviors, such as the ability to learn even when using AI tools. Typical questions included:

The questions were like:

- How would you approach learning about a project from a senior?
- How do you handle opposing viewpoints in a project discussion?
- How would you handle a new assignment using AI tools?
- How do you learn something totally new from scratch?
- How would you ramp up a project with the help of AI?

The trick is to show a genuine interest in learning and the company. I also heard people mention "spec-driven development," so it might be worth looking into: [Using spec-driven development with Claude Code](#)

FINAL ROUND: PEOPLE EXPERIENCE (PX)

- Where are you from?
- What is your opinion of Trimble?
- What do you know about Trimble?
- Why should we hire you?

Narrate your story in a way that aligns with the company's values while remaining calm and composed.

Keep smiling, maintain a positive attitude, and demonstrate your passion for learning. This was my experience, and yours may vary, so stay calm!

ALL THE BEST FOR YOUR INTERVIEW!